

Variant calling

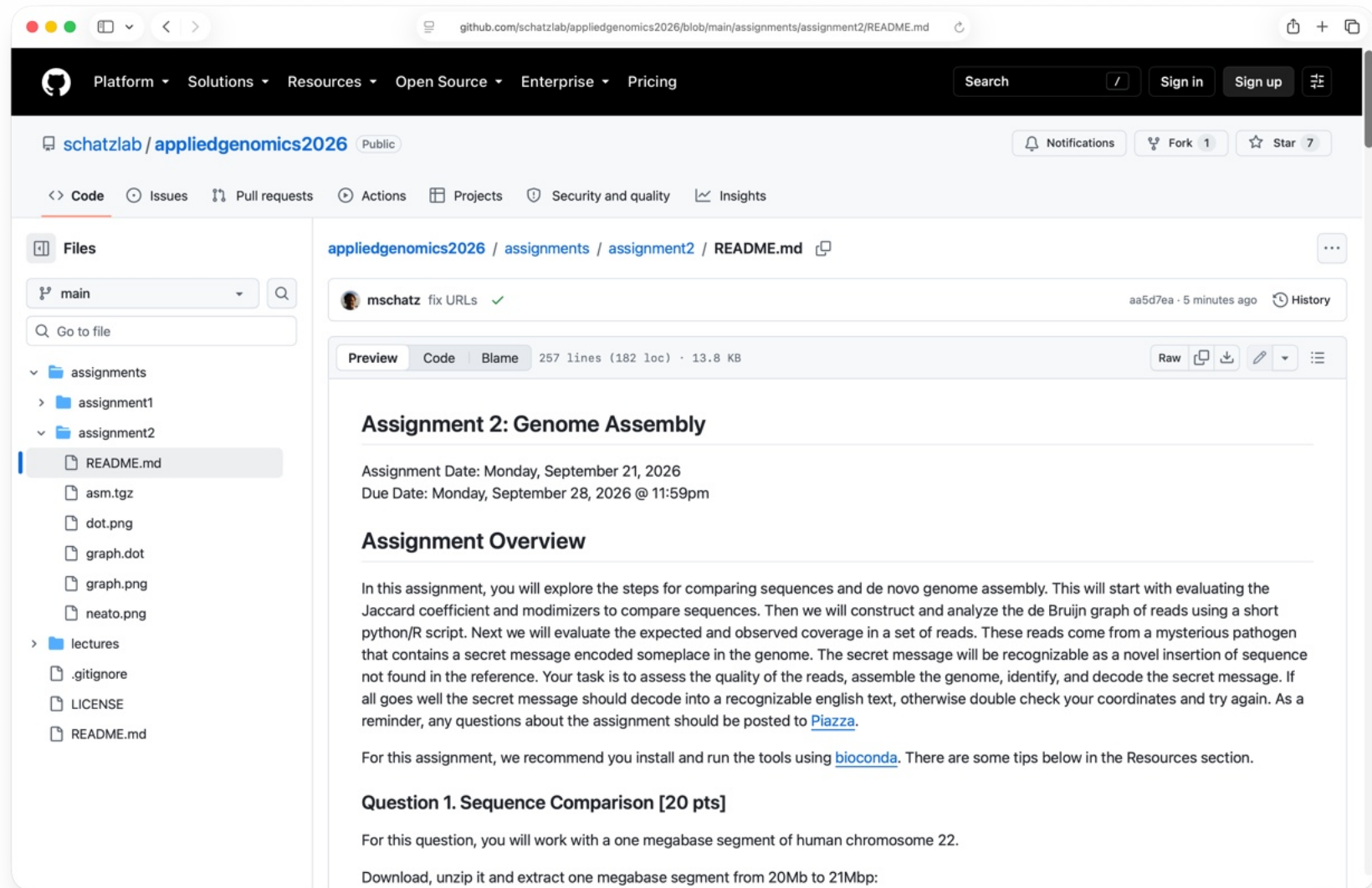
Michael Schatz

Sept 23, 2026

Lecture 7: Applied Comparative Genomics



Assignment 2



The screenshot shows a GitHub repository page for 'schatzlab / appliedgenomics2026'. The left sidebar shows the file structure with 'assignment2' selected, highlighting the 'README.md' file. The main content area shows the README file's content, which includes the assignment title, dates, an overview, and the first question.

Assignment 2: Genome Assembly

Assignment Date: Monday, September 21, 2026
Due Date: Monday, September 28, 2026 @ 11:59pm

Assignment Overview

In this assignment, you will explore the steps for comparing sequences and de novo genome assembly. This will start with evaluating the Jaccard coefficient and modimizers to compare sequences. Then we will construct and analyze the de Bruijn graph of reads using a short python/R script. Next we will evaluate the expected and observed coverage in a set of reads. These reads come from a mysterious pathogen that contains a secret message encoded someplace in the genome. The secret message will be recognizable as a novel insertion of sequence not found in the reference. Your task is to assess the quality of the reads, assemble the genome, identify, and decode the secret message. If all goes well the secret message should decode into a recognizable english text, otherwise double check your coordinates and try again. As a reminder, any questions about the assignment should be posted to [Piazza](#).

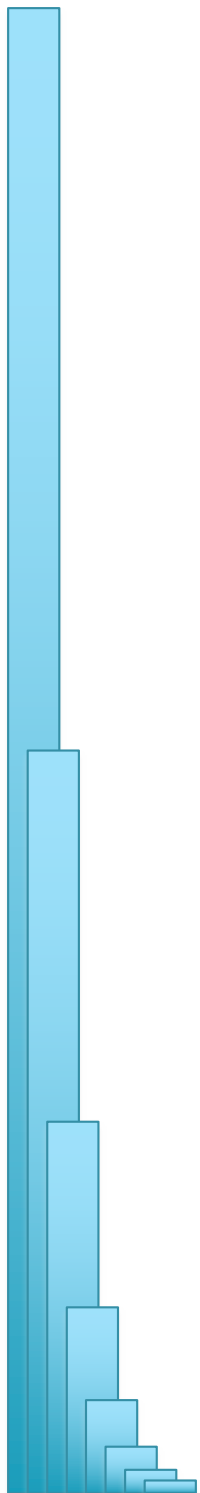
For this assignment, we recommend you install and run the tools using [bioconda](#). There are some tips below in the Resources section.

Question 1. Sequence Comparison [20 pts]

For this question, you will work with a one megabase segment of human chromosome 22.

Download, unzip it and extract one megabase segment from 20Mb to 21Mbp:

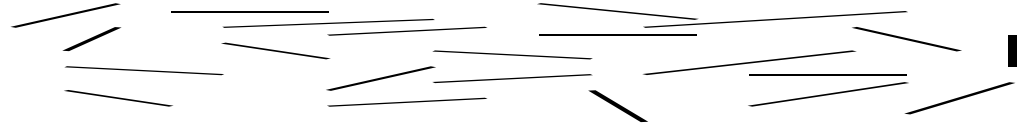
<https://github.com/schatzlab/appliedgenomics2026/tree/main/assignments/assignment2>
Due end of day on Monday Sept 28 (right before midnight)



0. Recap

Assembling a Genome

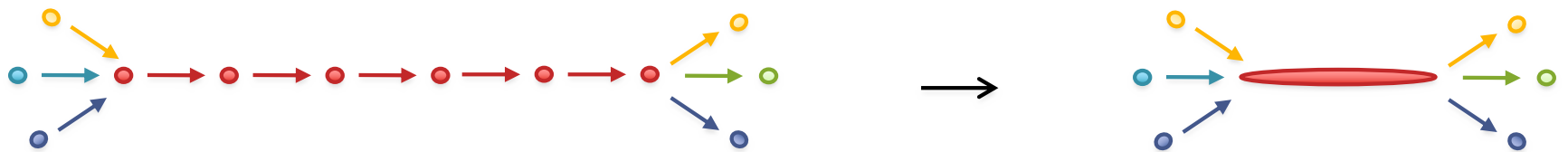
1. Shear & Sequence DNA



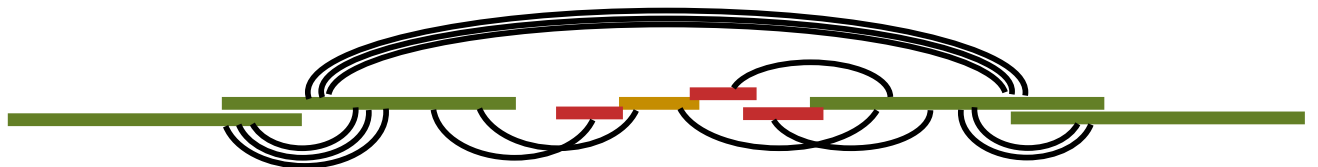
2. Construct assembly graph from reads (de Bruijn / overlap graph)

...AGCCTAGGGATGCGCGACACGT
GGATGCGCGACACGT CGCATATCCGGTTTGGT CAACCTCGGACGGAC
CAACCTCGGACGGACCTCAGCGAA...

3. Simplify assembly graph



4. Detangle graph with long reads, mates, and other links



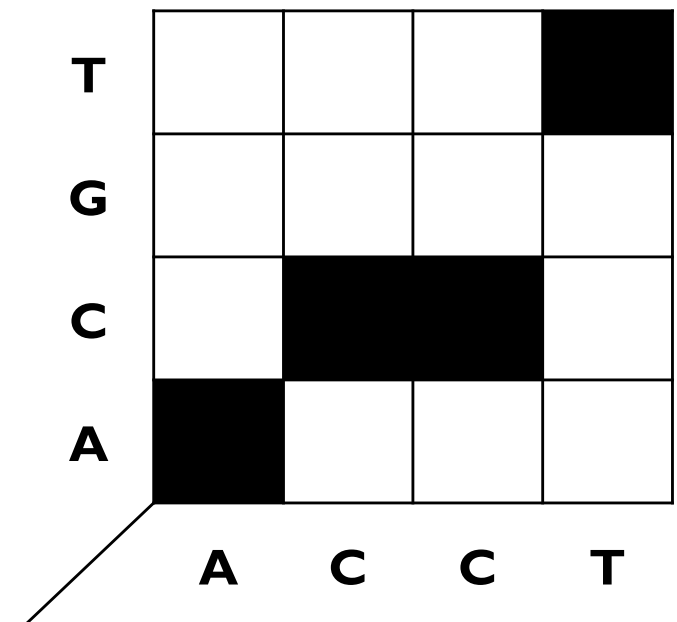
WGA visualization

- How can we visualize *whole* genome alignments?

- With an alignment dot plot

- $N \times M$ matrix

- Let i = position in genome A
 - Let j = position in genome B
 - Fill cell (i,j) if A_i shows similarity to B_j

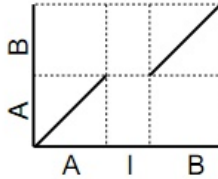


- A perfect alignment between A and B would completely fill the positive diagonal

SV Types

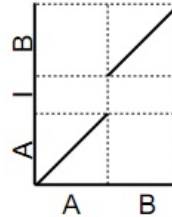
Insertion into Reference

R: AIB
Q: AB



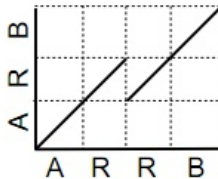
Insertion into Query

R: AB
Q: AIB



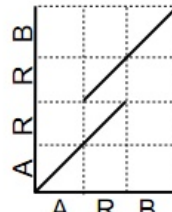
Collapse Query

R: ARRB
Q: ARB



Collapse Reference

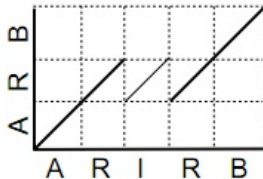
R: ARB
Q: ARRB



Collapse Query
w/ Insertion

R: ARIRB
Q: ARB

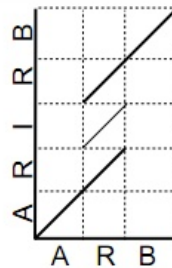
Exact tandem
alignment if I=R



Collapse Reference
w/ Insertion

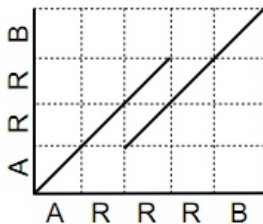
R: ARB
Q: ARIRB

Exact tandem
alignment if I=R



Collapse Query

R: ARRRB
Q: ARRB



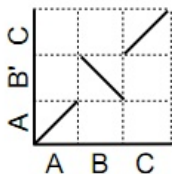
Collapse Reference

R: ARRB
Q: ARRRB



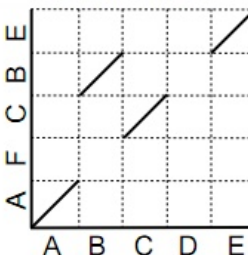
Inversion

R: ABC
Q: AB'C



Rearrangement
w/ Disagreement

R: ABCDE
Q: AFCBE



- Different structural variation types / misassemblies will be apparent by their pattern of breakpoints
- Most breakpoints will be at or near repeats
- Things quickly get complicated in real genomes

<http://mummer.sf.net/manual/AlignmentTypes.pdf>

GATTACATACACAATTGGATG
 GAT ACA ACA ATT GAT
 ATT CAT CAC TTG ATG
 TTA ATA ACA TGG
 TAC TAC CAT GGA

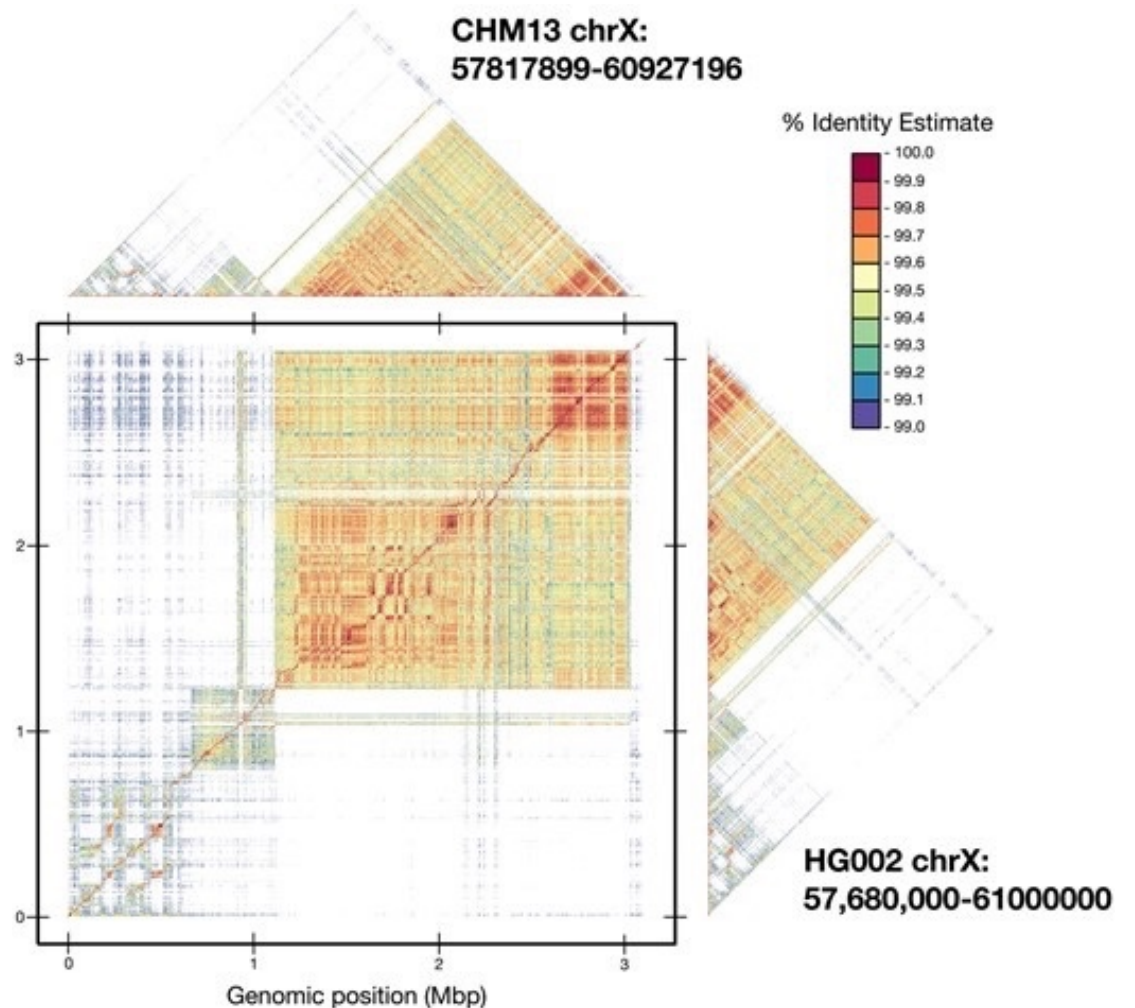
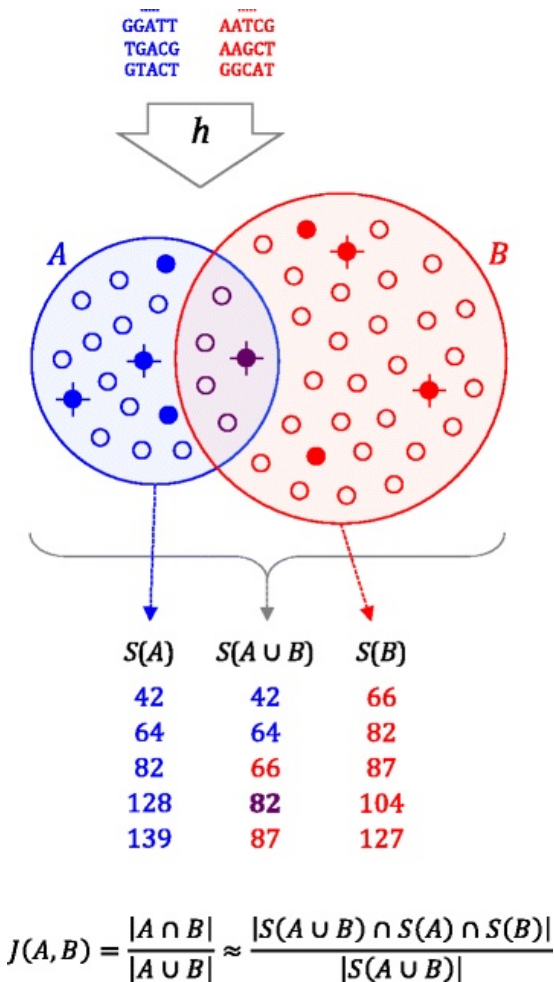
GATTACATACXCAATTGGATG
 GAT ACA ACX ATT GAT
 ATT CAT CX C TTG ATG
 TTA ATA XCA TGG
 TAC TAC CAT GGA



$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} = \frac{11}{15} = 73.3\%$$

$$\text{ANI} \approx 1 + \frac{1}{k} \ln \left(\frac{2J}{1+J} \right)$$

ANI = 94.4%

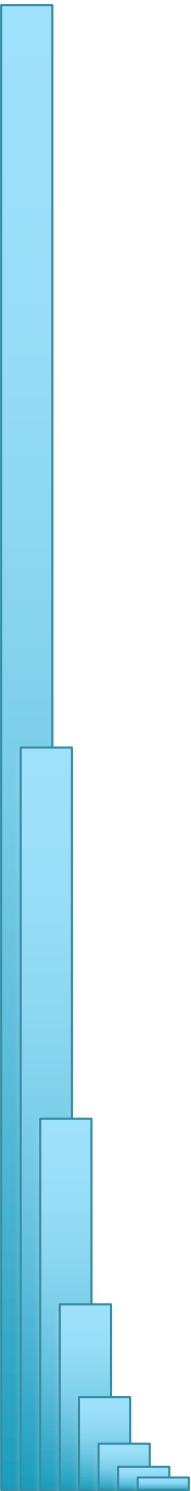


Mash: fast genome and metagenome distance estimation using MinHash

Ondov et al (2016) Genome Biology. <https://doi.org/10.1186/s13059-016-0997-x>

ModDotPlot—rapid and interactive visualization of tandem repeats

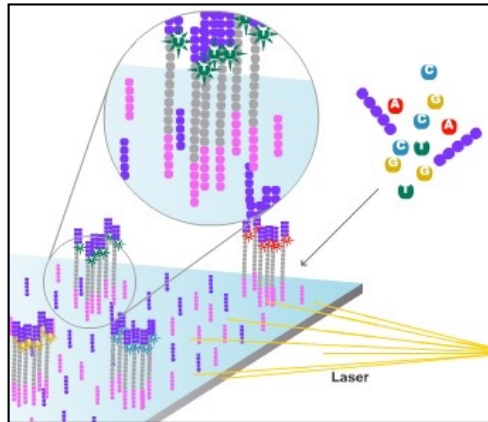
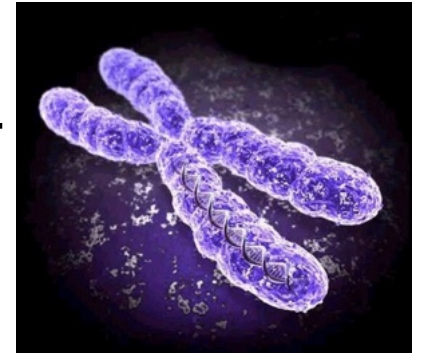
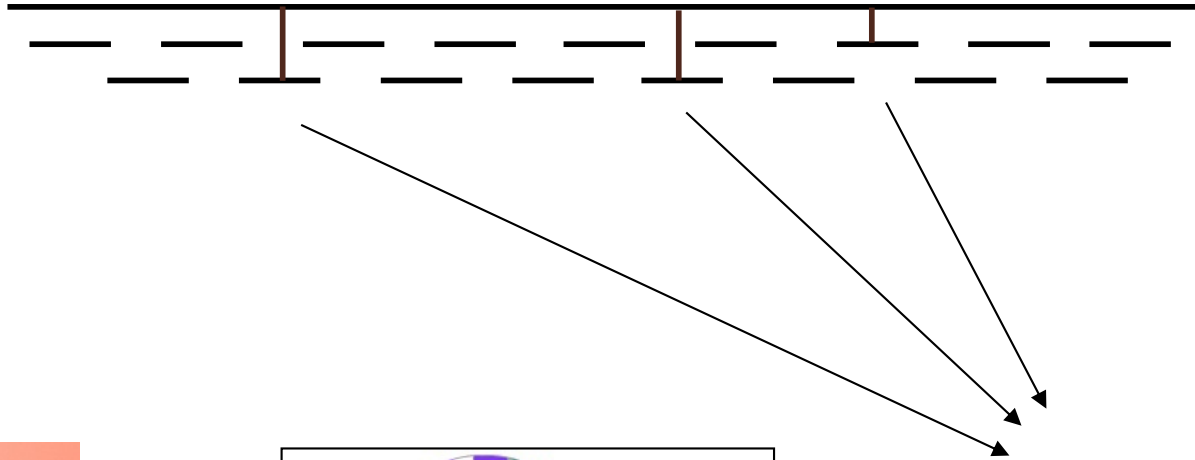
Sweeten, Schatz, Phillippy (2024) Bioinformatics. <https://doi.org/10.1093/bioinformatics/btae493>



Read Mapping & Variant calling

Personal Genomics

How does your genome compare to the reference?



Heart Disease

Cancer

Amazing actress



Variant Calling Overview



FASTQ Files



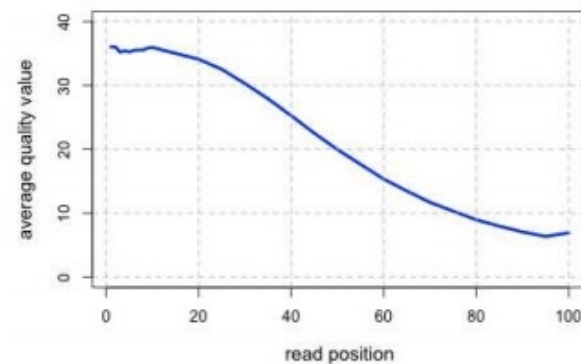
```
@SEQ_ID
GATTGTTGGGGTTCAAAGCAGTATCGATCAAATAGTAAATCCATTGTTCACACTCACAGTTT
+
!''*(((((***+))%%%++)(%%%)).1***-+*''))*55CCF>>>>>CCCCCCC65
```

@Identifier
Sequence
+Separator
Quality Values
...

Illumina Quality

QV	P _{error}
40	1/10000
30	1/1000
20	1/100
10	1/10

$$Q_{\text{sanger}} = -10 \log_{10} p$$



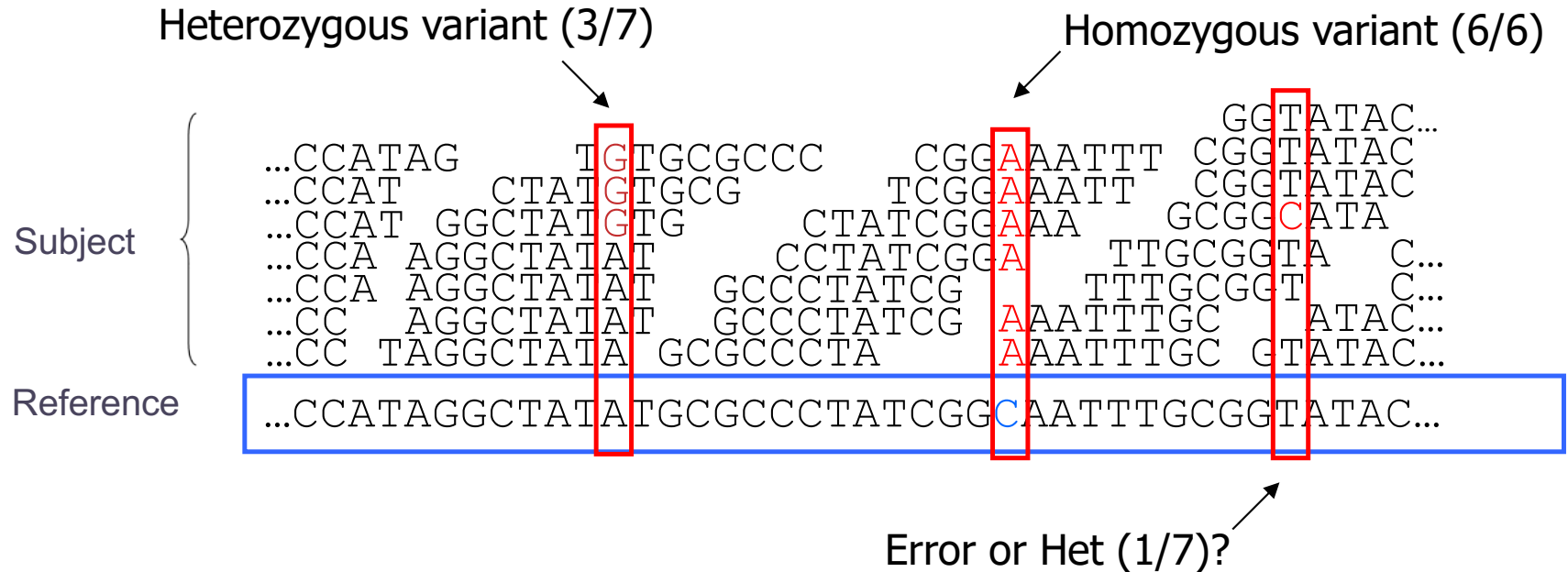
```
SSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSS.....  
.....XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX.....  
.....IIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIII.....  
.....JJJJJJJJJJJJJJJJJJJJJJJJJJJJJJJJJJJJ.....  
LLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLL.....  
!"#$%&'()*+,-./0123456789:;<=>?@ABCDEFGHIJKLMN  
OPQRSTUVWXYZ[\]^_`abcdefghijklmnopqrstuvwxyz{|}~
```

| | | | |

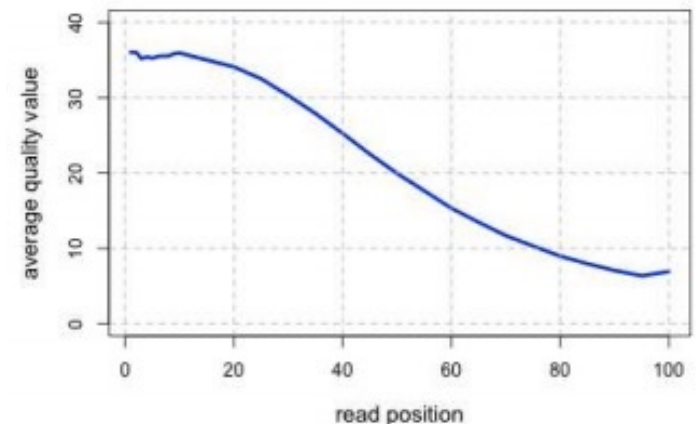
33 59 64 73 104 126

S - Sanger Phred+33, raw reads typically (0, 40)
X - Solexa Solexa+64, raw reads typically (-5, 40)
I - Illumina 1.3+ Phred+64, raw reads typically (0, 40)
J - Illumina 1.5+ Phred+64, raw reads typically (3, 40)
with 0=unused, 1=unused, 2=Read Segment Quality Control Indicator (bold)
(Note: See discussion above).
L - Illumina 1.8+ Phred+33, raw reads typically (0, 41)

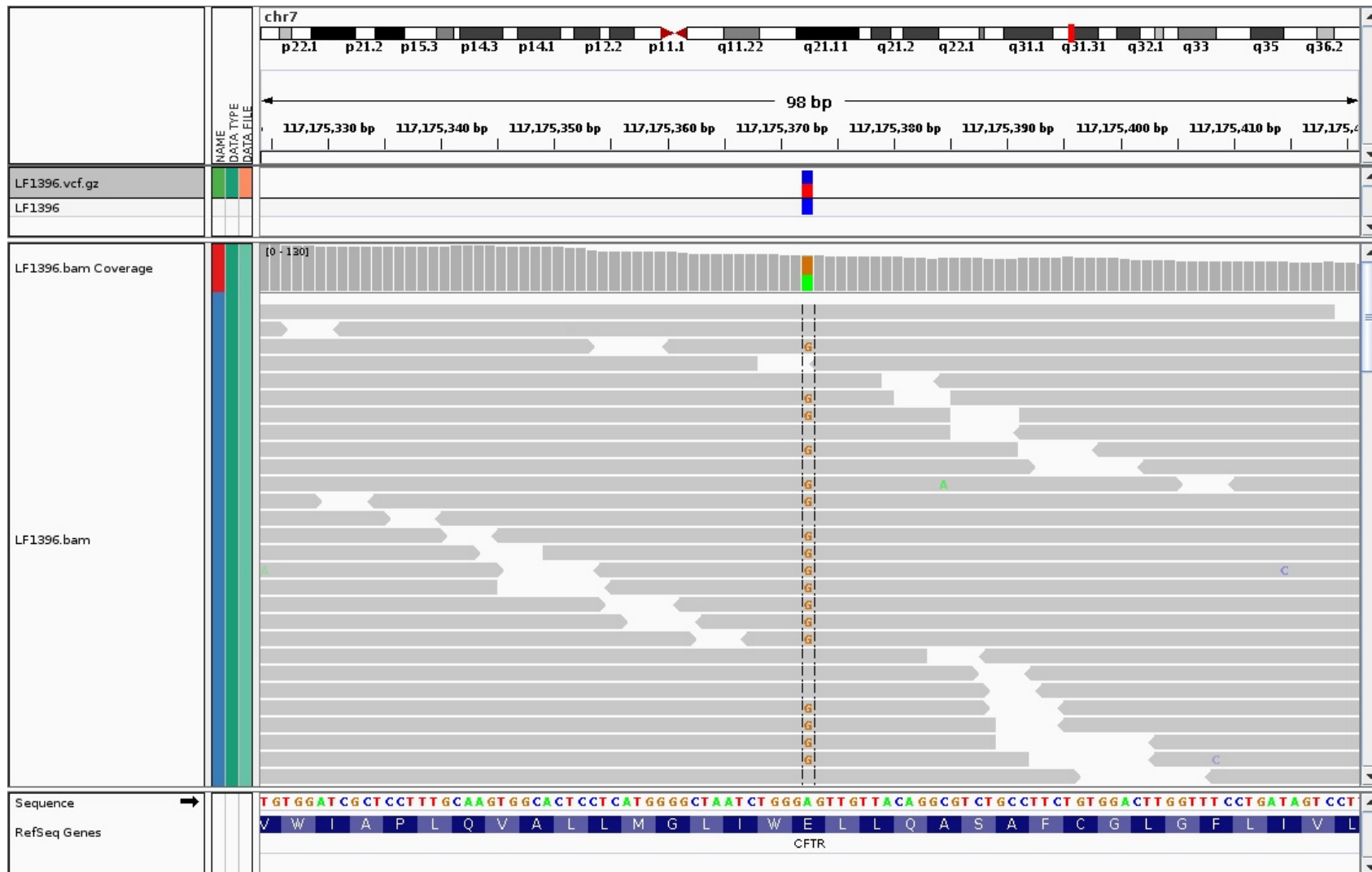
Genotyping Theory



- If there were no sequencing errors, identifying SNPs would be very easy: any time a read disagrees with the reference, it must be a variant!
- Sequencing instruments make mistakes
 - Quality of read decreases over the read length
- A single read differing from the reference is probably just an error, but it becomes more likely to be real as we see it multiple times



VCF Format



#CHROM	POS	ID	REF	ALT	QUAL	FILTER	INFO	FORMAT	LF1396
chr7	117175373	.	A	G	90	PASS	AF=0.5	GT	0/1

VCF Format

Example

VCF header

```
##fileformat=VCFv4.0
##fileDate=20100707
##source=VCFtools
##reference=NCBI36
##INFO=<ID=AA,Number=1,Type=String,Description="Ancestral Allele">
##INFO=<ID=H2,Number=0,Type=Flag,Description="HapMap2 membership">
##FORMAT=<ID=GT,Number=1,Type=String,Description="Genotype">
##FORMAT=<ID=GQ,Number=1,Type=Integer,Description="Genotype Quality (phred score)">
##FORMAT=<ID=GL,Number=3,Type=Float,Description="Likelihoods for RR,RA,AA genotypes (R=ref,A=alt)">
##FORMAT=<ID=DP,Number=1,Type=Integer,Description="Read Depth">
##ALT=<ID=DEL,Description="Deletion">
##INFO=<ID=SVTYPE,Number=1,Type=String,Description="Type of structural variant">
##INFO=<ID=END,Number=1,Type=Integer,Description="End position of the variant">
```

Mandatory header lines

Optional header lines (meta-data about the annotations in the VCF body)

Body

#CHROM	POS	ID	REF	ALT	QUAL	FILTER	INFO	FORMAT	SAMPLE1	SAMPLE2
1	1	.	ACG	A,AT	.	PASS	.	GT:DP	1/2:13	0/0:29
1	2	rs1	C	T,CT	.	PASS	H2;AA=T	GT:GQ	0/1:100	2/2:70
1	5	.	A	G	.	PASS	.	GT:GQ	1/0:77	1/1:95
1	100	.	T		.	PASS	SVTYPE=DEL;END=300	GT:GQ:DP	1/1:12:3	0/0:20

Deletion

SNP

Large SV

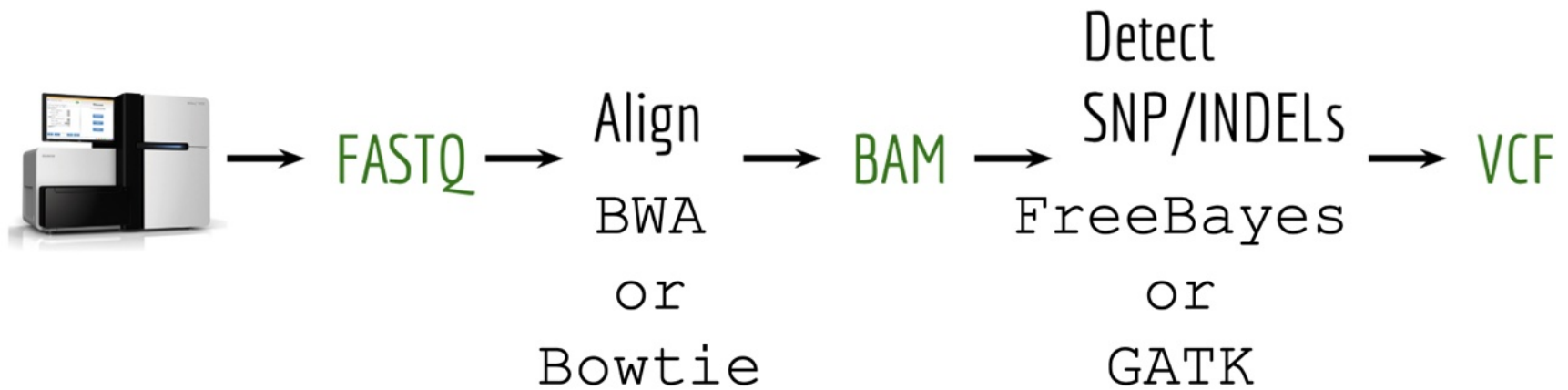
Insertion

Other event

Reference alleles (GT=0)

Alternate alleles (GT>0 is an index to the ALT column)

Phased data (G and C above are on the same chromosome)



Part I: Read Mapping

Brute Force Analysis



- Brute Force:
 - At every possible offset in the genome:
 - Do all of the characters of the query match?
- Analysis
 - Simple, easy to understand
 - Genome length = n [3B]
 - Query length = m [7]
 - Comparisons: $(n-m+1) * m$ [21B]
- Overall runtime: $O(nm)$
 - [How long would it take if we double the genome size, read length?]
 - [How long would it take if we double both?]

Brute Force Reflections

Why check every position?

- GATTACA can't possibly start at position 15

[WHY?]

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	...
T	G	A	T	T	A	C	A	G	A	T	T	A	C	C	...
								G	A	T	T	A	C	A	

- Improve runtime to $O(n + m)$

[3B + 7]

- If we double both, it just takes twice as long
- Knuth-Morris-Pratt, 1977
- Boyer-Moyer, 1977, 1991

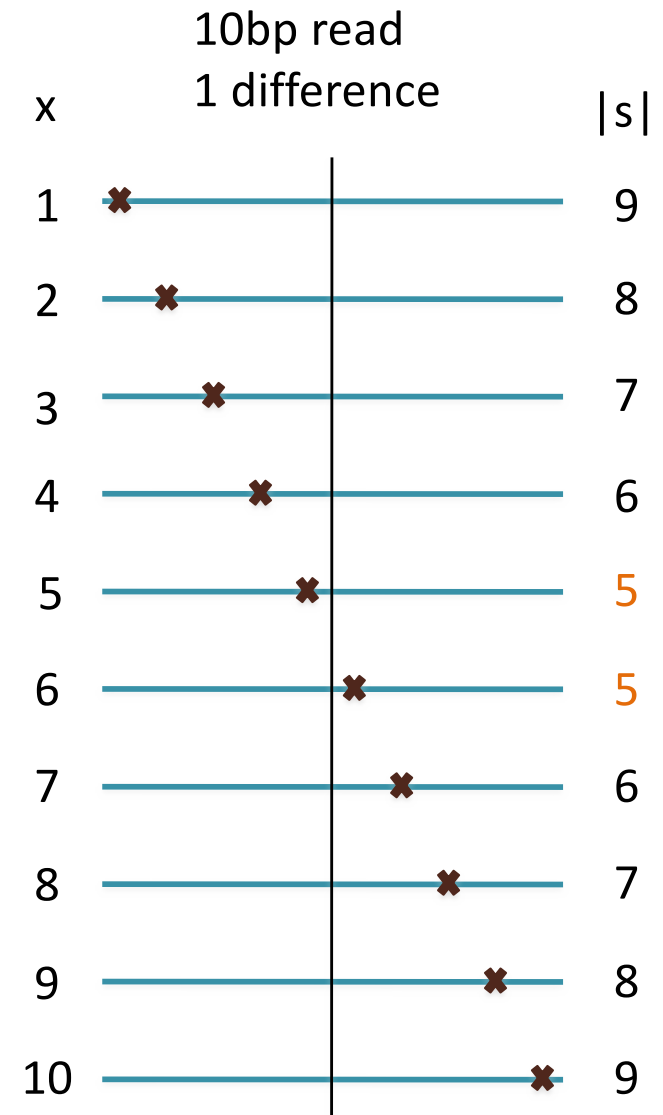
- For one-off scans, this is the best we can do (optimal performance)

- We have to read every character of the genome, and every character of the query
- For short queries, runtime is dominated by the length of the genome

Seed-and-Extend Alignment

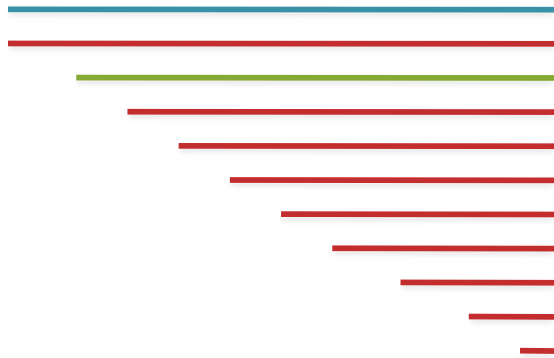
Theorem: An alignment of a sequence of length m with at most k differences **must** contain an exact match at least $s = m / (k + 1)$ bp long
(Baeza-Yates and Perleberg, 1996)

- Proof: Pigeonhole principle
 - 1 pigeon can't fill 2 holes
- Seed-and-extend search
 - Use an index to rapidly find short exact alignments to seed longer in-exact alignments
 - BLAST, MUMmer, Bowtie, BWA, SOAP, ...
 - Specificity of the depends on seed length
 - Guaranteed sensitivity for k differences
 - Also finds some (but not all) lower quality alignments <- heuristic

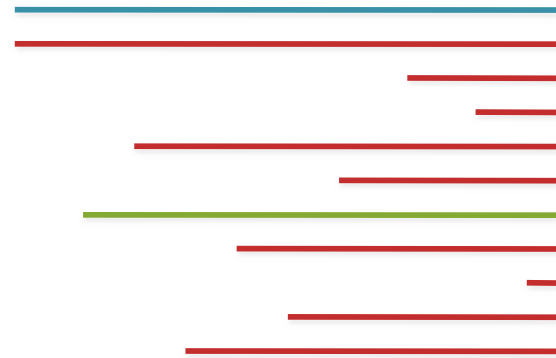


Suffix Arrays: Searching the Phone Book

- What if we need to check many queries?
 - We don't need to check every page of the phone book to find 'Schatz'
 - Sorting alphabetically lets us immediately skip 96% (25/26) of the book *without any loss in accuracy*
- Sorting the genome: Suffix Array (Manber & Myers, 1991)
 - Sort every suffix of the genome



Split into n suffixes



Sort suffixes alphabetically

[Challenge Question: How else could we split the genome?]

Searching the Index

- Strategy 2: Binary search
 - Compare to the middle, refine as higher or lower
- Searching for GATTACA
 - Lo = 1; Hi = 15;

Lo
→

#	Sequence	Pos
1	ACAGATTACC...	6
2	ACC...	13
3	AGATTACC...	8
4	ATTACAGATTACC...	3
5	ATTACC...	10
6	C...	15
7	CAGATTACC...	7
8	CC...	14
9	GATTACAGATTACC...	2
10	GATTACC...	9
11	TACAGATTACC...	5
12	TACC...	12
13	TGATTACAGATTACC...	1
14	TTACAGATTACC...	4
15	TTACC...	11

Hi
→

Searching the Index

- Strategy 2: Binary search
 - Compare to the middle, refine as higher or lower
- Searching for GATTACA
 - $Lo = 1; Hi = 15; Mid = (1+15)/2 = 8$
 - Middle = Suffix[8] = CC

Lo
→

#	Sequence	Pos
1	ACAGATTACC...	6
2	ACC...	13
3	AGATTACC...	8
4	ATTACAGATTACC...	3
5	ATTACC...	10
6	C...	15
7	CAGATTACC...	7
8	CC...	14
9	GATTACAGATTACC...	2
10	GATTACC...	9
11	TACAGATTACC...	5
12	TACC...	12
13	TGATTACAGATTACC...	1
14	TTACAGATTACC...	4
15	TTACC...	11

Hi
→

Searching the Index

- Strategy 2: Binary search
 - Compare to the middle, refine as higher or lower
- Searching for GATTACA
 - $Lo = 1; Hi = 15; Mid = (1+15)/2 = 8$
 - Middle = Suffix[8] = CC
=> Higher: $Lo = Mid + 1$

Lo
→

#	Sequence	Pos
1	ACAGATTACC...	6
2	ACC...	13
3	AGATTACC...	8
4	ATTACAGATTACC...	3
5	ATTACC...	10
6	C...	15
7	CAGATTACC...	7
8	CC...	14
9	GATTACAGATTACC...	2
10	GATTACC...	9
11	TACAGATTACC...	5
12	TACC...	12
13	TGATTACAGATTACC...	1
14	TTACAGATTACC...	4
15	TTACC...	11

Hi
→

Searching the Index

- Strategy 2: Binary search
 - Compare to the middle, refine as higher or lower
- Searching for GATTACA
 - $Lo = 1; Hi = 15; Mid = (1+15)/2 = 8$
 - Middle = Suffix[8] = CC
=> Higher: $Lo = Mid + 1$
 - $Lo = 9; Hi = 15;$

#	Sequence	Pos
1	ACAGATTACC...	6
2	ACC...	13
3	AGATTACC...	8
4	ATTACAGATTACC...	3
5	ATTACC...	10
6	C...	15
7	CAGATTACC...	7
8	CC...	14
9	GATTACAGATTACC...	2
10	GATTACC...	9
11	TACAGATTACC...	5
12	TACC...	12
13	TGATTACAGATTACC...	1
14	TTACAGATTACC...	4
15	TTACC...	11

Lo
→

Hi
→

Searching the Index

- Strategy 2: Binary search
 - Compare to the middle, refine as higher or lower
- Searching for GATTACA
 - $Lo = 1; Hi = 15; Mid = (1+15)/2 = 8$
 - $Middle = Suffix[8] = CC$
=> Higher: $Lo = Mid + 1$
 - $Lo = 9; Hi = 15; Mid = (9+15)/2 = 12$
 - $Middle = Suffix[12] = TACC$

#	Sequence	Pos
1	ACAGATTACC...	6
2	ACC...	13
3	AGATTACC...	8
4	ATTACAGATTACC...	3
5	ATTACC...	10
6	C...	15
7	CAGATTACC...	7
8	CC...	14
9	GATTACAGATTACC...	2
10	GATTACC...	9
11	TACAGATTACC...	5
12	TACC...	12
13	TGATTACAGATTACC...	1
14	TTACAGATTACC...	4
15	TTACC...	11

Lo
→

Hi
→

Searching the Index

- Strategy 2: Binary search
 - Compare to the middle, refine as higher or lower
- Searching for GATTACA
 - $Lo = 1; Hi = 15; Mid = (1+15)/2 = 8$
 - $Middle = Suffix[8] = CC$
=> Higher: $Lo = Mid + 1$
 - $Lo = 9; Hi = 15; Mid = (9+15)/2 = 12$
 - $Middle = Suffix[12] = TACC$
=> Lower: $Hi = Mid - 1$
 - $Lo = 9; Hi = 11;$

#	Sequence	Pos
1	ACAGATTACC...	6
2	ACC...	13
3	AGATTACC...	8
4	ATTACAGATTACC...	3
5	ATTACC...	10
6	C...	15
7	CAGATTACC...	7
8	CC...	14
9	GATTACAGATTACC...	2
10	GATTACC...	9
11	TACAGATTACC...	5
12	TACC...	12
13	TGATTACAGATTACC...	1
14	TTACAGATTACC...	4
15	TTACC...	11

Lo
→

Hi
→

Searching the Index

- Strategy 2: Binary search
 - Compare to the middle, refine as higher or lower
- Searching for GATTACA
 - $Lo = 1; Hi = 15; Mid = (1+15)/2 = 8$
 - $Middle = Suffix[8] = CC$
=> Higher: $Lo = Mid + 1$
 - $Lo = 9; Hi = 15; Mid = (9+15)/2 = 12$
 - $Middle = Suffix[12] = TACC$
=> Lower: $Hi = Mid - 1$
 - $Lo = 9; Hi = 11; Mid = (9+11)/2 = 10$
 - $Middle = Suffix[10] = GATTACC$

#	Sequence	Pos
1	ACAGATTACC...	6
2	ACC...	13
3	AGATTACC...	8
4	ATTACAGATTACC...	3
5	ATTACC...	10
6	C...	15
7	CAGATTACC...	7
8	CC...	14
9	GATTACAGATTACC...	2
10	GATTACC...	9
11	TACAGATTACC...	5
12	TACC...	12
13	TGATTACAGATTACC...	1
14	TTACAGATTACC...	4
15	TTACC...	11

Lo
→

Hi
→

Searching the Index

- Strategy 2: Binary search
 - Compare to the middle, refine as higher or lower
- Searching for GATTACA
 - $Lo = 1; Hi = 15; Mid = (1+15)/2 = 8$
 - $Middle = Suffix[8] = CC$
=> Higher: $Lo = Mid + 1$
 - $Lo = 9; Hi = 15; Mid = (9+15)/2 = 12$
 - $Middle = Suffix[12] = TACC$
=> Lower: $Hi = Mid - 1$
 - $Lo = 9; Hi = 11; Mid = (9+11)/2 = 10$
 - $Middle = Suffix[10] = GATTACC$
=> Lower: $Hi = Mid - 1$
 - $Lo = 9; Hi = 9;$

#	Sequence	Pos
1	ACAGATTACC...	6
2	ACC...	13
3	AGATTACC...	8
4	ATTACAGATTACC...	3
5	ATTACC...	10
6	C...	15
7	CAGATTACC...	7
8	CC...	14
9	GATTACAGATTACC...	2
10	GATTACC...	9
11	TACAGATTACC...	5
12	TACC...	12
13	TGATTACAGATTACC...	1
14	TTACAGATTACC...	4
15	TTACC...	11

Lo
Hi
→

Searching the Index

- Strategy 2: Binary search
 - Compare to the middle, refine as higher or lower
- Searching for GATTACA
 - $Lo = 1; Hi = 15; Mid = (1+15)/2 = 8$
 - $Middle = Suffix[8] = CC$
=> Higher: $Lo = Mid + 1$
 - $Lo = 9; Hi = 15; Mid = (9+15)/2 = 12$
 - $Middle = Suffix[12] = TACC$
=> Lower: $Hi = Mid - 1$
 - $Lo = 9; Hi = 11; Mid = (9+11)/2 = 10$
 - $Middle = Suffix[10] = GATTACC$
=> Lower: $Hi = Mid - 1$
 - $Lo = 9; Hi = 9; Mid = (9+9)/2 = 9$
 - $Middle = Suffix[9] = GATTACA...$
=> Match at position 2!

#	Sequence	Pos
1	ACAGATTACC...	6
2	ACC...	13
3	AGATTACC...	8
4	ATTACAGATTACC...	3
5	ATTACC...	10
6	C...	15
7	CAGATTACC...	7
8	CC...	14
9	GATTACAGATTACC...	2
10	GATTACC...	9
11	TACAGATTACC...	5
12	TACC...	12
13	TGATTACAGATTACC...	1
14	TTACAGATTACC...	4
15	TTACC...	11

Lo
Hi
→

Binary Search Analysis

- Binary Search

Initialize search range to entire list

$\text{mid} = (\text{hi} + \text{lo}) / 2$; $\text{middle} = \text{suffix}[\text{mid}]$

if query matches middle: done

else if query < middle: pick low range

else if query > middle: pick hi range

Repeat until done or empty range

[WHEN?]

- Analysis

- More complicated method

- How many times do we repeat?

- How many times can it cut the range in half?

- Find smallest x such that: $n / (2^x) \leq 1$; $x = \lg_2(n)$

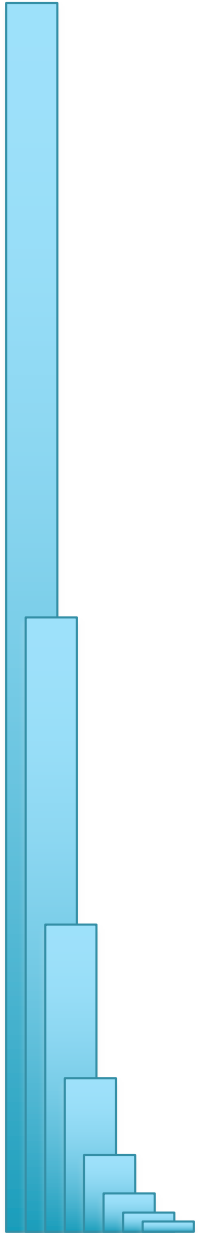
[32]

- Total Runtime: $O(m \lg n)$

- More complicated, but **much** faster!

- Looking up a query loops 32 times instead of 3B

[How long does it take to search 6B or 24B nucleotides?]



Binary Search Analysis

- Binary Search

Initialize search range to entire list

$mid = (hi+lo)/2$; $middle = suffix[mid]$

if query matches middle: done

else if query < middle: pick low range

else if query > middle: pick hi range

Repeat until done or empty range

[WHEN?]

- Analysis

- More complicated method

- How many times do we repeat?

- How many times can it cut the range in half?

- Find smallest x such that: $n/(2^x) \leq 1$; $x = \lg_2(n)$

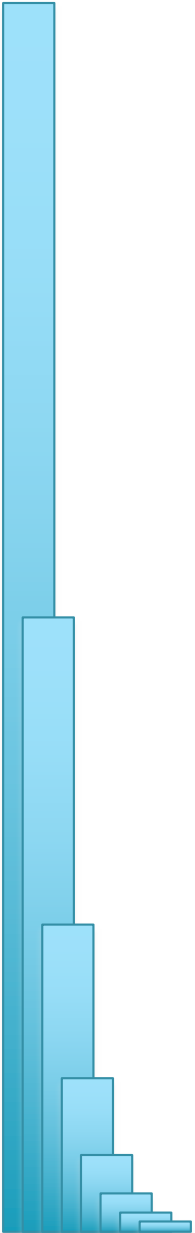
[32]

- Total Runtime: $O(m \lg n)$

- More complicated, but **much** faster!

- Looking up a query loops 32 times instead of 3B

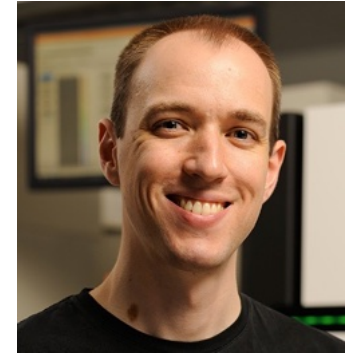
Can be reduced to $O(m + \lg n)$
using an auxiliary data structure called the LCP array



Algorithmic challenge

How can we combine the speed of a suffix array $O(m + \lg(n))$ (or even $O(m)$) with the size of a brute force analysis (n bytes)?

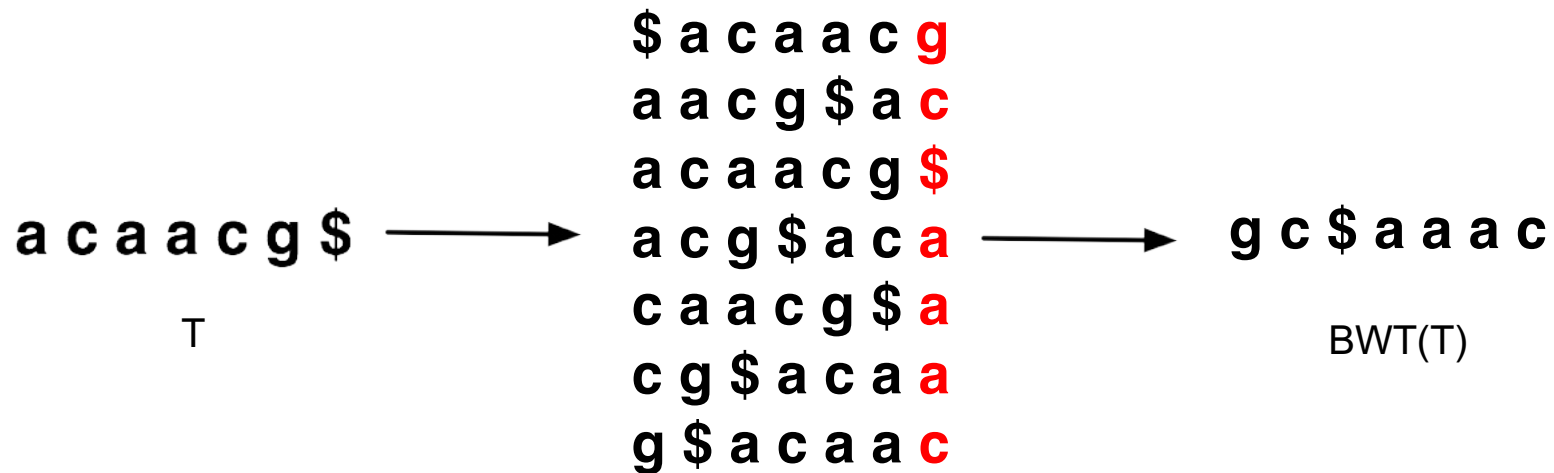
What would such an index look like?



**Bowtie: Ultrafast and memory
efficient alignment of short DNA
sequences to the human genome**

Burrows-Wheeler Transform

- Permutation of the characters in a text



BWT(T) is the index for T

A block sorting lossless data compression algorithm.

Burrows M, Wheeler DJ (1994) *Digital Equipment Corporation*. Technical Report 124

Algorithm Overview

1. Split read into segments

Read Read (reverse complement)

CCAGTAGCTCTCAGCCTTATTTTACCCAGGCCTGTA TACAGGCCTGGGTAAAATAAGGCTGAGAGCTACTGG

Policy: extract 16 nt seed every 10 nt

Seeds

+, 0: CCAGTAGCTCTCAGCC	-, 0: TACAGGCCTGGGTAAA
+, 10: TCAGCCTTATTTTACC	-, 10: GGTAAAATAAGGCTGA
+, 20: TTTACCCAGGCCTGTA	-, 20: GGCTGAGAGCTACTGG

2. Lookup each segment and prioritize

[illegible]

3. Evaluate end-to-end match

Extension candidates

SA:684, chr12:1955
SA:624, chr2:462
SA:211, chr4:762
SA:213, chr12:1935
SA:652, chr12:1945

→

SIMD dynamic programming aligner

→

SAM alignments

```
r1    0      chr12     1936      0
      36M *   0       0
      CCAGTAGCTCTCAGCCTTATTTTACCCAGGCCTGTA
      I::::::::::::::::::::::::::::::::::::
AS:i:0   XS:i:-2   XN:i:0
XM:i:0   XO:i:0   XG:i:0
NM:i:0   MD:Z:36  YT:Z:UU
YM:i:0
```

(Langmead & S)

(Langmead & Salzberg, 2012)

Similarity metrics

- Hamming distance

- Count the number of substitutions to transform one string into another

MIKESCHATZ

| | X | | XXXX |

MICESHATZZ

5

- Edit distance

- The minimum number of substitutions, insertions, or deletions to transform one string into another

MIKESCHAT-Z

| | X | | X | | | X |

MICES-HATZZ

3

Edit Distance Example

AGCACACA → ACACACTA in 4 steps

AGCACACA → (1. change G to C)

ACCACACA → (2. delete C)

ACACACA → (3. change A to T)

ACACACT → (4. insert A after T)

ACACACTA → done

[Is this the best we can do?]

Edit Distance Example

AGCACACA → ACACACTA in 3 steps

AGCACACA → (1. change G to C)

ACCACACA → (2. delete C)

ACACACA → (3. insert T after 3rd C)

ACACACTA → done

[Is this the best we can do?]

Reverse Engineering Edit Distance

$$D(\text{AGCACACA}, \text{ACACACTA}) = ?$$

Imagine we already have the optimal alignment of the strings, the last column can only be 1 of 3 options:

...M	...I	...D
...A	...-	...A
...A	...A	...-

The optimal alignment of last two columns is then 1 of 9 possibilities

...MM	...IM	...DM	...MI	...II	...DI	...MD	...ID	...DD
...CA	...-A	...CA	...A-	...--	...A-	...CA	...-A	...CA
...TA	...TA	...-A	...TA	...TA	...-A	...A-	...A-	...--

The optimal alignment of the last three columns is then 1 of 27 possibilities...

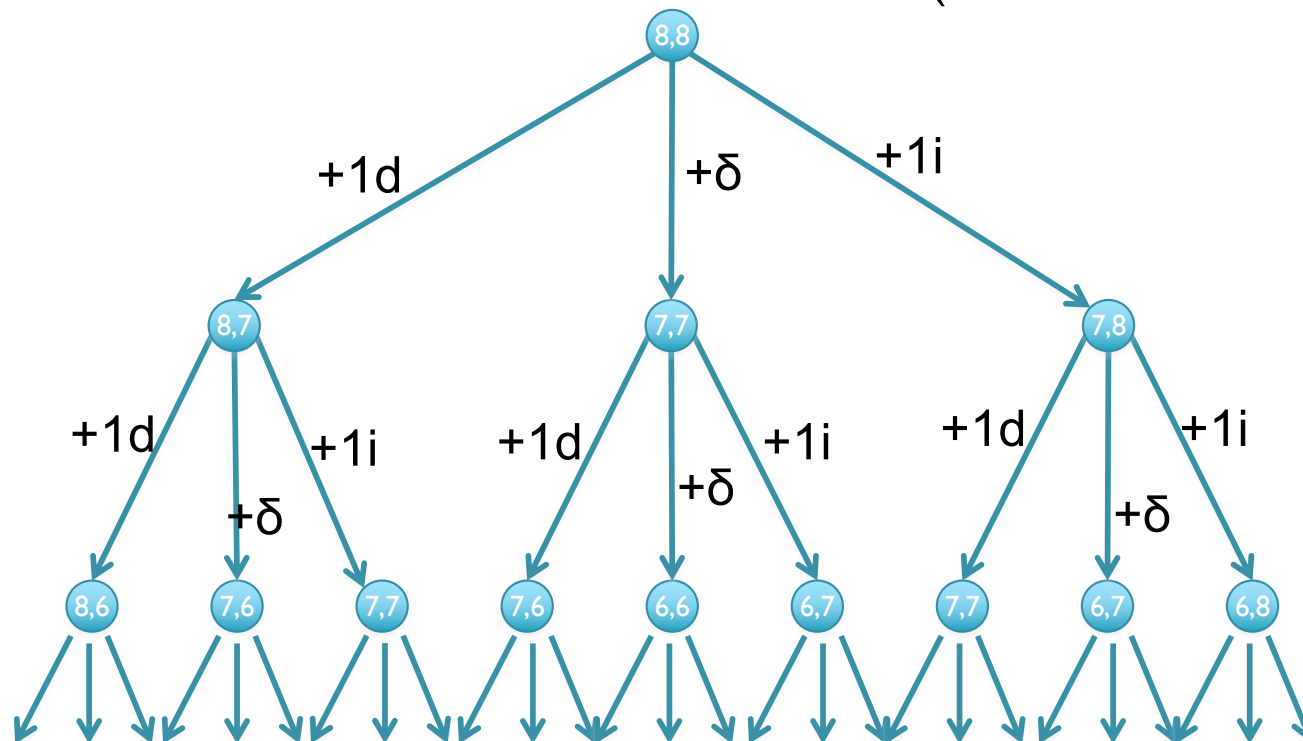
...M...	...I...	...D...
...X...	...-...	...X...
...Y...	...Y...	...-...

Eventually spell out every possible sequence of {I,M,D}

Recursive solution

- Computation of D is a recursive process.
 - At each step, we only allow matches, substitutions, and indels
 - $D(i,j)$ in terms of $D(i',j')$ for $i' \leq i$ and $j' \leq j$.

$$D(\text{AGCACACA}, \text{ACACACTA}) = \min \{ D(\text{AGCACACA}, \text{ACACACT}) + 1, \\ D(\text{AGCACAC}, \text{ACACACTA}) + 1, \\ D(\text{AGCACAC}, \text{ACACACT}) + \delta(\text{A}, \text{A}) \}$$



[What is the running time?]

Dynamic Programming

- We could code this as a recursive function call...
...with an exponential number of function evaluations
- There are only $(n+1) \times (m+1)$ pairs i and j
 - We are evaluating $D(i,j)$ multiple times
- Compute $D(i,j)$ bottom up.
 - Start with smallest $(i,j) = (1,1)$.
 - Store the intermediate results in a table.
 - Compute $D(i,j)$ *after* $D(i-1,j)$, $D(i,j-1)$, and $D(i-1,j-1)$

Recurrence Relation for D

Find the edit distance (minimum number of operations to convert one string into another) in $O(mn)$ time

- Base conditions:

- $D(i,0) = i$, for all $i = 0,\dots,n$
- $D(0,j) = j$, for all $j = 0,\dots,m$

- For $i > 0, j > 0$:

$$D(i,j) = \min \left\{ \begin{array}{ll} D(i-1,j) + 1, & // \text{ align 0 chars from S, 1 from T} \\ D(i,j-1) + 1, & // \text{ align 1 char from S, 0 from T} \\ D(i-1,j-1) + \delta(S(i),T(j)) & // \text{ align 1+1 chars} \end{array} \right\}$$

[Why do we want the min?]

Dynamic Programming Matrix

		A	C	A	C	A	C	T	A
	0	1	2	3	4	5	6	7	8
A	1								
G	2								
C	3								
A	4								
C	5								
A	6								
C	7								
A	8								

[What does the initialization mean?]

Dynamic Programming Matrix

		A	C	A	C	A	C	T	A
	0	1	2	3	4	5	6	7	8
A	1	0							
G	2								
C	3								
A	4								
C	5								
A	6								
C	7								
A	8								

$$D[A,A] = \min\{D[A,]+1, D[,A]+1, D[,]+ \delta(A,A)\}$$

Dynamic Programming Matrix

		A	C	A	C	A	C	T	A
	0	1	2	3	4	5	6	7	8
A	1	0	1						
G	2								
C	3								
A	4								
C	5								
A	6								
C	7								
A	8								

$$D[A,AC] = \min\{D[A,A]+1, D[,AC]+1, D[,A]+\delta(A,C)\}$$

Dynamic Programming Matrix

		A	C	A	C	A	C	T	A
	0	1	2	3	4	5	6	7	8
A	1	0	1	2					
G	2								
C	3								
A	4								
C	5								
A	6								
C	7								
A	8								

$$D[A,ACA] = \min\{D[A,AC]+1, D[,ACA]+1, D[,AC]+\delta(A,A)\}$$

Dynamic Programming Matrix

		A	C	A	C	A	C	T	A
	<u>0</u>	<u>1</u>	<u>2</u>	<u>3</u>	<u>4</u>	<u>5</u>	<u>6</u>	<u>7</u>	<u>8</u>
A	1	0	1	2	3	4	5	6	<u>7</u>
G	2								
C	3								
A	4								
C	5								
A	6								
C	7								
A	8								

$$D[A, ACACACTA] = 7$$

-----A

*****|

ACACACTA

[What about the other A?]

Dynamic Programming Matrix

		A	C	A	C	A	C	T	A
	<u>0</u>	<u>1</u>	<u>2</u>	<u>3</u>	<u>4</u>	5	6	7	8
A	1	0	1	2	3	<u>4</u>	5	6	7
G	2	1	1	2	3	4	<u>5</u>	<u>6</u>	<u>7</u>
C	3								
A	4								
C	5								
A	6								
C	7								
A	8								

$D[AG, ACACACTA] = 7$

-----AG--

***** | ***

ACACACTA

Dynamic Programming Matrix

		A	C	A	C	A	C	T	A
	<u>0</u>	1	2	3	4	5	6	7	8
A	1	<u>0</u>	1	2	3	4	5	6	7
G	2	<u>1</u>	1	2	3	4	5	6	7
C	3	2	<u>1</u>	2	2	3	4	5	6
A	4	3	2	<u>1</u>	2	2	3	4	5
C	5	4	3	2	<u>1</u>	2	2	3	4
A	6	5	4	3	2	<u>1</u>	2	3	3
C	7	6	5	4	3	2	<u>1</u>	<u>2</u>	3
A	8	7	6	5	4	3	2	2	<u>2</u>

$$D[\text{AGCACACA}, \text{ACACACTA}] = 2$$

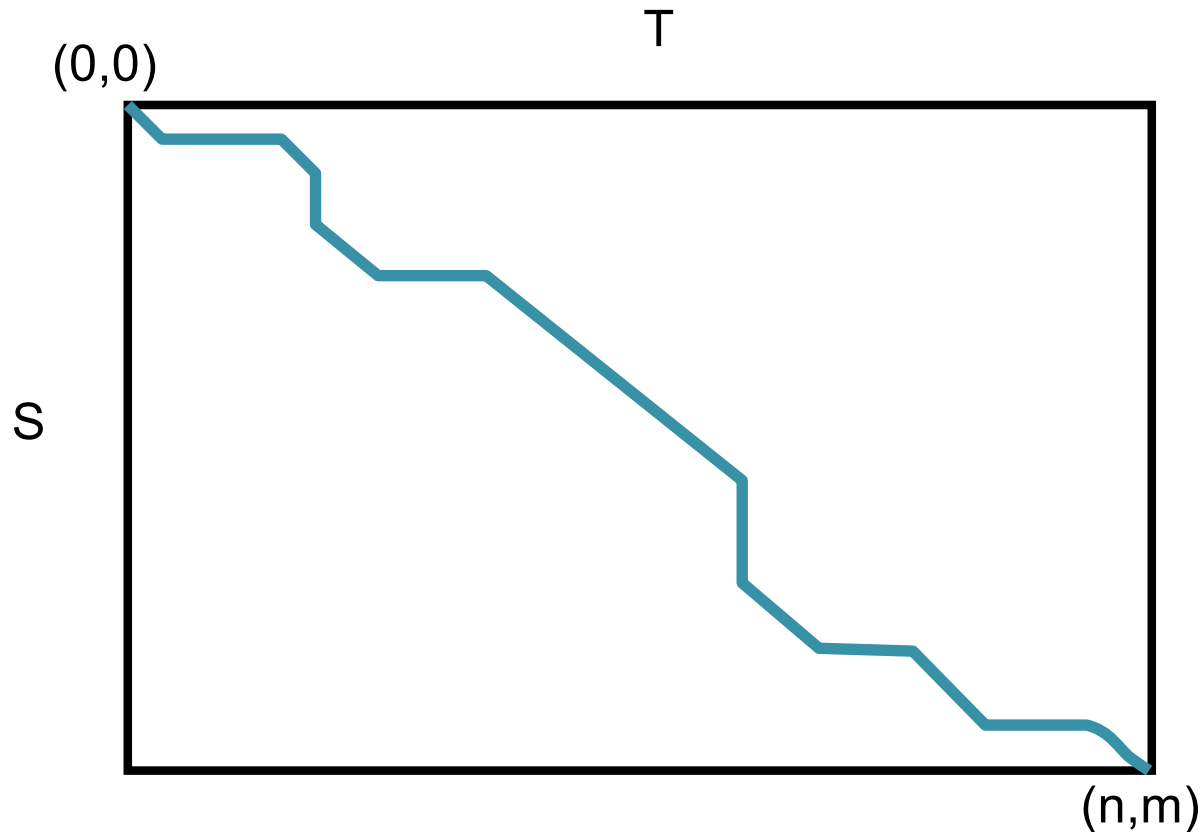
AGCACAC-A

|*| | | | |*|

A-CACACTA

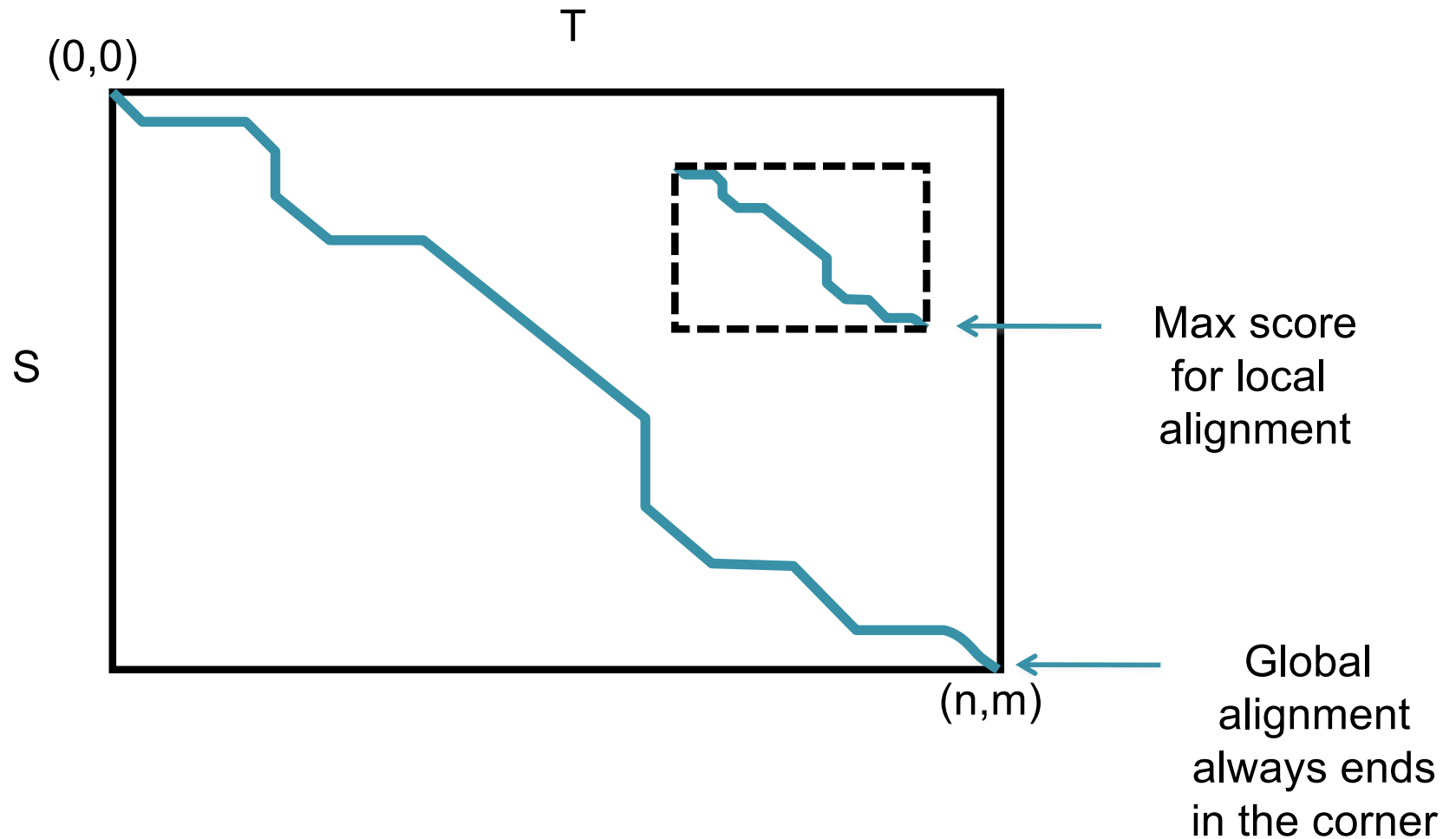
[Can we do it any better?]

Global Alignment Schematic



- A high quality alignment will stay close to the diagonal
 - If we are only interested in high quality alignments, we can skip filling in cells that can't possibly lead to a high quality alignment
 - Find the global alignment with at most edit distance d : $O(2dn)$

Global vs Local Alignment Schematic



SAM / BAM Files

```
Coord      12345678901234 5678901234567890123456789012345
ref         AGCATGTTAGATAA**GATAGCTGTGCTAGTAGGCAGTCAGCGCCAT

+r001/1      TTAGATAAAGGATA*CTG
+r002        aaaAGATAA*GGATA
+r003        gcctaAGCTAA
+r004                ATAGCT.....TCAGC
-r003                ttagctTAGGC
-r001/2                        CAGCGGCAT
```

```
@HD VN:1.6 SO:coordinate
@SQ SN:ref LN:45
r001 99 ref 7 30 8M2I4M1D3M = 37 39 TTAGATAAAGGATACTG *
r002 0 ref 9 30 3S6M1P1I4M * 0 0 AAAAGATAAGGATA *
r003 0 ref 9 30 5S6M * 0 0 GCCTAAGCTAA * SA:Z:ref,29,-,6H5M,17,0;
r004 0 ref 16 30 6M14N5M * 0 0 ATAGCTTCAGC *
r003 2064 ref 29 17 6H5M * 0 0 TAGGC * SA:Z:ref,9,+,5S6M,30,1;
r001 147 ref 37 30 9M = 7 -39 CAGCGGCAT * NM:i:1
```

SAM / BAM Files

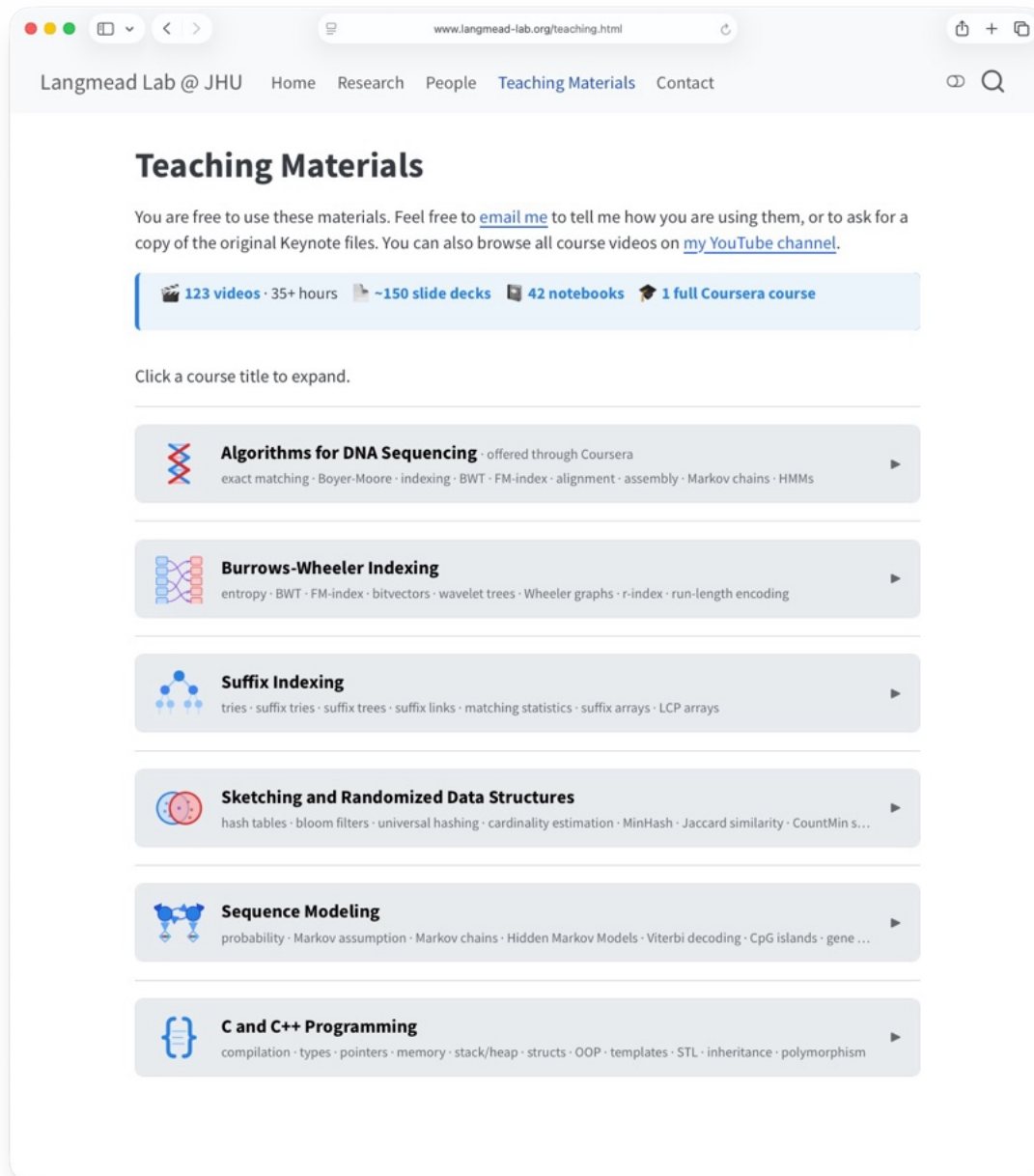
1.4 The alignment section: mandatory fields

In the SAM format, each alignment line typically represents the linear alignment of a segment. Each line consists of 11 or more TAB-separated fields. The first eleven fields are always present and in the order shown below; if the information represented by any of these fields is unavailable, that field's value will be a placeholder, either '0' or '*' as determined by the field's type. The following table gives an overview of these mandatory fields in the SAM format:

Col	Field	Type	Regexp/Range	Brief description
1	QNAME	String	[!-?A-~]{1,254}	Query template NAME
2	FLAG	Int	[0, 2 ¹⁶ - 1]	bitwise FLAG
3	RNAME	String	* [:rname:^*=] [:rname:]*	Reference sequence NAME ¹¹
4	POS	Int	[0, 2 ³¹ - 1]	1-based leftmost mapping POSition
5	MAPQ	Int	[0, 2 ⁸ - 1]	MAPping Quality
6	CIGAR	String	* ([0-9]+[MIDNSHPX=])+	CIGAR string
7	RNEXT	String	* = [:rname:^*=] [:rname:]*	Reference name of the mate/next read
8	PNEXT	Int	[0, 2 ³¹ - 1]	Position of the mate/next read
9	TLEN	Int	[-2 ³¹ + 1, 2 ³¹ - 1]	observed Template LENgth
10	SEQ	String	* [A-Za-z=.]+	segment SEquence
11	QUAL	String	[!-~]+	ASCII of Phred-scaled base QUALity+33

¹¹Reference sequence names may contain any printable ASCII characters with the exception of certain punctuation characters, and may not start with '*' or '='. See Section 1.2.1 for details and an explanation of the [:rname:] notation.

Ben Langmead's Course



601.447
COMPUTATIONAL
GENOMICS: SEQUENCES

Topics will include: string matching, sequence alignment and indexing, assembly, and sequence models. Course will involve significant programming projects.